1. Aim

To analyse the State Electricity Subsidy Eligibility rules, implement them using separate if statements and an if–elif–else chain, compare their outputs, and select a clear decision structure.

2. Problem Description

The subsidy rules are:

- Consumption of 100 units or less: full subsidy.
- Consumption from 101 to 200 units: partial subsidy only when the household has a ration card.
- Consumption above 200 units: no subsidy unless the household has a member above 80 years of age.

The decision must be based on consumption, ration-card status and the presence of a household member above 80 years old.

3. Predictions Before Execution

Eight test cases are selected to cover the required boundary values (100, 101, 200 and 201) and different household conditions.

Case	Units	Ration card	Member above 80	Predicted decision
1	80	No	No	Full subsidy
2	100	Yes	No	Full subsidy
3	101	Yes	No	Partial subsidy
4	150	No	No	No subsidy
5	200	Yes	No	Partial subsidy
6	201	Yes	No	No subsidy
7	250	No	Yes	Special subsidy for senior member
8	250	Yes	No	No subsidy

4. Implementation 1 – Separate if Statements

Each if statement is evaluated independently. A list is used so that the program can record every matching decision.

```
def subsidy_separate(units, ration_card, senior_member):
    decisions = []

    if units <= 100:
        decisions.append("Full subsidy")

    if 101 <= units <= 200 and ration_card:
        decisions.append("Partial subsidy")

    if units > 200:
        if senior_member:
            decisions.append("Special subsidy for senior member")
        else:
            decisions.append("No subsidy")

    if not decisions:
        decisions.append("No subsidy")

    return decisions
```

```
tests = [
    (80, False, False),
    (100, True, False),
    (101, True, False),
    (150, False, False),
    (200, True, False),
    (201, True, False),
    (250, False, True),
    (250, True, False)
]

for case in tests:
    print(case, "->", subsidy_separate(*case))
```

For the exact non-overlapping ranges used here, only one consumption range applies. However, separate if statements do not prevent later conditions from running. If conditions overlap or are written too broadly, more than one result can be appended. The list makes multiple matches visible rather than hiding them.

5. Implementation 2 – if–elif–else

The if–elif–else chain checks conditions in order. Once a branch is selected, the remaining branches are skipped.

```
def subsidy_chain(units, ration_card, senior_member):
    if units <= 100:
        return "Full subsidy"
    elif 101 <= units <= 200:
        if ration_card:
            return "Partial subsidy"
        else:
            return "No subsidy"
    else:  # units > 200
        if senior_member:
            return "Special subsidy for senior member"
        else:
            return "No subsidy"


for case in tests:
    print(case, "->", subsidy_chain(*case))
```

This version returns exactly one decision for every valid non-negative consumption value. The inner if–else handles the additional conditions within the relevant consumption range.

6. Comparison of Results

Case	Separate-if result	if–elif–else result	Comparison
1	Full subsidy	Full subsidy	Same
2	Full subsidy	Full subsidy	Same
3	Partial subsidy	Partial subsidy	Same
4	No subsidy	No subsidy	Same
5	Partial subsidy	Partial subsidy	Same
6	No subsidy	No subsidy	Same
7	Special subsidy for senior member	Special subsidy for senior member	Same
8	No subsidy	No subsidy	Same

For these eight test cases, both implementations produce the same intended decision. The key difference is structural: separate if statements are independent, whereas an if–elif–else chain selects one branch and prevents later branches from executing.

7. Why Multiple Results Can Occur

With separate if statements, Python tests every condition independently. If two conditions are true, both blocks execute. For example, if a programmer accidentally writes if units >= 100 for one rule and if units <= 200 for another, a value such as 150 satisfies both conditions. An if–elif–else chain executes the first matching branch only, so it prevents multiple branch results.

The given subsidy slabs are mutually exclusive when their boundaries are coded correctly. Therefore, separate if statements do not automatically produce two subsidies in this particular implementation; the risk arises when conditions overlap, are too broad, or are changed incorrectly.

8. Design Decision

Combined Boolean conditions with a clear if–elif–else structure are the best choice for this problem. The consumption ranges are mutually exclusive and form a simple sequence. The ration-card and senior-member checks are nested only inside the applicable consumption range. This keeps the rules readable and makes future changes to one slab easier to locate.

Deeply nested conditions would add indentation and make the main consumption ranges harder to scan. Combining every condition into one long Boolean expression could also make the rule difficult to understand. A short chain with small nested checks is clearer.

9. Alternative Concept – match–case

Python 3.10 and later supports match–case. Since the rules depend on ranges and additional Boolean conditions, guards can be used:

```
def subsidy_match(units, ration_card, senior_member):
    match units:
        case n if n <= 100:
            return "Full subsidy"
        case n if 101 <= n <= 200 and ration_card:
            return "Partial subsidy"
        case n if 101 <= n <= 200:
            return "No subsidy"
        case n if n > 200 and senior_member:
            return "Special subsidy for senior member"
        case _:
            return "No subsidy"
```

The guards are checked in order. The final wildcard case provides a default decision. This approach is possible, but it is not necessarily clearer than if–elif–else for simple numeric ranges.

10. Alternative Concept – Lookup List and Loop

A lookup design can store the upper limit and decision label in a list. A loop searches the slabs in order. Additional conditions are then checked for the selected slab.

```
def subsidy_lookup(units, ration_card, senior_member):
    slabs = [
        (100, "FULL"),
        (200, "MIDDLE"),
        (float("inf"), "ABOVE")
    ]

    for upper_limit, slab in slabs:
        if units <= upper_limit:
```

```
if slab == "FULL":
    return "Full subsidy"
elif slab == "MIDDLE" and ration_card:
    return "Partial subsidy"
elif slab == "MIDDLE":
    return "No subsidy"
elif slab == "ABOVE" and senior_member:
    return "Special subsidy for senior member"
else:
    return "No subsidy"
```

This separates the slab boundaries from the decisions. It can be useful when there are many slabs or when slab data changes frequently. For only three slabs, it adds more code than the if–elif–else solution.

11. Time and Space Complexity

Approach	Time per decision	Extra space	Explanation
if–elif–else	O(1)	O(1)	A fixed number of conditions is checked; no data structure is required.
match–case with guards	O(1)	O(1)	A fixed number of cases is checked in order.
Lookup list and loop	O(k)	O(k)	In the worst case, the loop checks k slabs; the list stores k entries.

Here, k is the number of subsidy slabs. If the number of slabs is fixed at three, all approaches take constant time in practice. If k grows, the lookup loop takes O(k) time and O(k) space for its stored table. The if–elif–else and guarded match–case approaches use O(1) extra space, although a long chain may also require checking many conditions.

12. Conclusion

Both implementations gave the predicted results for all eight test cases, including the four boundary values. The if–elif–else version is preferred because it communicates that exactly one decision should be made. Nested checks keep the ration-card and senior-member rules within the relevant consumption ranges. A lookup list is a reasonable alternative for a large, frequently changing set of slabs, but it is unnecessary for this small rule set.

13. Execution Evidence

Run the Python program and insert the actual console output or a screenshot here. The predictions and comparison table above are the expected results; execution evidence must come from running the submitted code.